Undersigned: talking points for Alyssa

Fri 30 Oct · MS3 due Thu 12 Nov · each item is what we wanted, what we tried, what happened

IF SHE SAYS "UPDATE ME"

1. **The MS3 stack is built and live.** The model trains on Vertex, every run is tracked in W&B, a Vertex pipeline only promotes a model that beats the one being served, and it's serving on Cloud Run with API tests in CI.
2. **The model ties Haiku for free, and its top 5 is as good as Sonnet's.** Its first pick is right 69.8% of the time (Haiku 69.0%, Sonnet 75.4%), and the model we serve has the right section in its top 5 93.1% of the time, the same as Sonnet. Sending only its least-sure passages to Sonnet gets 95% of Sonnet's accuracy at an eighth of the cost.
3. **It works on a rule it was never trained on.** We tested on the Department of Education's 2022 Title IX proposed rule (ED). Trained with zero letters about it, the model got 50%. Adding ED's own preamble to training (no labels needed) lifted it to 67%, close to the 73% it gets when it has trained on ED letters. Livia is running the other five rules this week.
4. **Next: beat Sonnet on the first pick.** The right answer is already in our top 3 85% of the time, so the job is choosing among 3, not 30. The idea we're most excited about: **the agency already labelled the comments for us.** When a rule is finalized, the agency publishes its answer to every significant comment, sorted by section. That's thousands of free, expert labels written by the people who read every letter. More ideas below. Also left for MS3: Sonnet call in the API, screenshots, slides.

WHAT THE ACCURACY NUMBERS MEAN

Each passage is ranked against every section of its rule (11 to 39 sections). **Top-1** = our first pick is the section the commenter cited. **Top-3 / top-5** = the cited section is somewhere in our first 3 / 5.

	Top-1	Top-3	Top-5
Our model, all 490 test passages	66.7%	85.1%	90.8%
Our model, the 232 Sonnet also answered	69.8%		92.2%
Served model (no extra signals), same 232	68.5%		93.1%
Sonnet, same 232	75.4%		93.1%
Plain search (MS2), same 232	53.4%		84.9%

So the model finds the right neighbourhood as well as Sonnet does. The gap is only in picking the single best one, which is what the next steps target. The report only has top-5 on the 232 subset, hence the blanks.

How it's set up

Comment letter → Split into passages, form-letter copies set aside → **Fine-tuned encoder** turns each passage into a vector → Compared to every section of the rule (long sections cut into windows, best window counts) → Sections ranked, with a margin. Small margin = flagged as unsure

THE MODEL, WHAT KIND OF LEARNING

- `BAAI/bge-small-en-v1.5`, a small text encoder, the same one MS2 already served. We fine-tune it; the tuned copy replaces the old one, no new service.
- **Supervised contrastive learning with labels we get for free.** Every example has a right answer (the section), so it isn't unsupervised. But no person labels anything: the labels come from the data itself, so it's closest to **self-supervised**.
- Contrastive: pull a passage's vector toward its section and push it away from wrong ones. The wrong ones are "hard negatives", the 3 wrong sections the model currently likes most, re-picked every epoch.

WHERE THE LABELS COME FROM

- Some commenters cite a section ("§ 106.44"). We hide the citation and make the model recover the section from what the passage argues.
- Plus the agency's preamble: each paragraph discussing a section becomes an example for that section, numbers hidden.
- 6,066 training examples, 299 validation. Test is a frozen set of 490 passages from 6 rules, none sharing text with training letters.

"UNSURE" AND SENDING TO SONNET

- Each section gets a similarity score. **Margin = top score minus second score.** Tiny margin means two sections are nearly tied.
- Margin tracks correctness: biggest-margin fifth 96% right (Sonnet 87%), smallest-margin fifth 33% (Sonnet 67%).
- **Cascade result is simulated:** Sonnet already answered these test passages in our 11 Oct test, with cost saved. We keep our answer when sure and swap in Sonnet's when unsure. Share sent is picked on half the comments, scored on the other half.
- **Live API doesn't call Sonnet yet.** It returns the margin and a `needs_reader` flag (26% flagged on the LCRI check). Calling Sonnet on flagged ones is a small next step.

VERTEX AI: TRAINING AND PIPELINE

- Training is a Vertex custom job: image built in Cloud Build, reads data from the bucket, writes the run back. CPU (n2-standard-16), since GPUs are blocked on the free trial.
- Vertex pipeline: **train → evaluate → promote**. It only promotes if the new model beats plain search and the one being served. This one passed, +12.9 over search.
- Same seed and data gave identical answers on the laptop and on Vertex, all 490 test passages.

W&B: WHAT WE TRACK

- Every training run logs loss, learning rate and validation accuracy per epoch, plus a table of test predictions. Config and commit are saved with the run.
- Project `undersigned-linker` on the team's W&B (CoreWeave host). Current model is run `ls2ekysw`, pipeline run `zyqws7hu`.
- It already caught a bug: validation sat flat at 73.9% for three epochs, which showed the learning-rate schedule hit zero halfway. Fixed, buggy run kept for comparison.

CLOUD RUN: HOW WE SERVE IT

- Service `undersigned-api` (FastAPI), us-central1, 2 CPU / 4 GB, scales 0 to 2 instances. Model exported to ONNX, so no PyTorch in the image.
- Deploy script builds in Cloud Build and bakes in the promoted model from the bucket. Live revision serves `in-pre`.
- Endpoints: `POST /link`, `/retrieve`, rule sections and passages, `/health`, `/stats`. JSON request logs. 172 ms typical response.
- Private right now: callers need a Google identity token. Making it public is a team decision.

WHAT IT COSTS ONCE WE ACTUALLY CALL SONNET (SERVED MODEL, MEASURED ON THE 232 PASSAGES SONNET ANSWERED)

Share of passages sent to Sonnet	Right first	$ per 1,000 passages	All 721k passages in the 7 test rules
0% (our model only)	68.5%	$0	$0
10%	70.7%	$1.80	$1,298
20% (what the API flags now)	73.3%	$3.50	$2,524
30%	75.4%	$5.77	$4,161
100% (Sonnet on everything)	75.4%	$19.58	$14,121

Sonnet's cost per passage is what it actually cost on 11 Oct. Only flagged passages are sent, so we pay roughly in proportion to the share. Fixed shares are scored on all 232 passages, which flatters them a little: with the share picked on one half and scored on the other, the honest number is 72.0% at $2.43. At 30% we match Sonnet for less than a third of its cost.

1 Linking comments to the right section

WE WANTED

MS2 could only place comments that cite a section number, and only 3.7% do. We wanted a model that reads any passage and says which section of the rule it's arguing about.

WE TRIED

Fine-tuning a small embedding model (bge-small). Labels come free: take comments that do cite, hide the citation, make the model guess it. Trained as a Vertex job, logged in W&B.

RESULT

65.5% right on the first guess over 490 test passages. Adding two extra signals (the agency's own discussion of each section, and other sections the letter cites) brought it to 66.7%. On 40 hand-labelled passages with no citation: 90%.

2 How it compares to an LLM

WE WANTED

To know whether a free small model is good enough, or whether we just need to pay for Claude. Same 232 passages for everyone.

RESULT

Method	Right first	$ per 1,000
Plain search (MS2)	53.4%	0
Our model	**69.8%**	**0**
Haiku	69.0%	1.29
Sonnet	75.4%	19.58

We tie Haiku and are about 6 points behind Sonnet.

3 Getting close to Sonnet without paying for Sonnet

WE WANTED

Sonnet-level accuracy at a cost we could afford on the full data. Running Sonnet on all 721k passages would cost $14,121.

WE TRIED

A cascade. Our model answers when it's confident and hands the unsure ones to Sonnet. This works because its confidence means something: on its surest fifth it beats Sonnet (96% vs 87%), on its least sure fifth it loses badly (33% vs 67%).

RESULT

72.0% at $2.43 per 1,000, which is 95% of Sonnet's accuracy for an eighth of the price. On all 721k passages: about $1,750 instead of $14,121. A cheaper setting gets 70.7% for about $960.

4 Rules the model has never seen

WE WANTED

ED = the Department of Education's 2022 Title IX proposed rule (sex discrimination in schools, docket ED-2021-OCR-0166, 19 sections). We wanted the model to work on a brand-new rule like this, since every campaign is about a different rule and nobody will have labelled letters for it.

WE TRIED

Training a copy with every ED letter removed, then testing on ED's 100 test passages. Then the same, but adding ED's preamble (the agency's own explanation of each section, which every proposed rule ships with) to training.

RESULT

No ED letters: 50% top-1. With ED's preamble: **67%** top-1, 87% top-3. With ED letters too: 73%. Plain search: 57%. So most of the gap closes with no labels at all.

5 Putting it in production

WE WANTED

The MS3 stack: Vertex training, W&B, a Vertex pipeline, and the model served on Cloud Run.

WE TRIED

A pipeline that trains, evaluates, and only replaces the live model if the new one beats both plain search and the current model. Then deployed to Cloud Run with logging and a `/stats` page.

RESULT

The new model passed (+12.9 points over search) and is live. Live check on 80 LCRI passages: 57.5% right first, 88.8% in the top 5, 172 ms typical response. 44 API tests run in CI.

6 GPU training

WE WANTED

To train on GPUs, which would also cover the optional multi-GPU part of MS3.

WE TRIED

Requesting 2 T4 GPUs in us-central1 on 29 Oct.

RESULT

Google blocked it because the project is on a free-trial billing account. It needs the billing owner to upgrade to paid. So far we've trained on CPU, which has been fine.

★ Ideas to float: beating Sonnet on the first pick

Idea	Why it could work	How we'd test it
1. The agency already labelled the comments. Train on the final rule's response to comments.	When a rule is finalized, the agency summarizes what commenters argued, section by section, and answers it. Those summaries are paraphrases of exactly our passages, labelled by the people who read every letter. Free, expert, and in commenter language, not legal language.	Parse the final rule's comment summaries for ED (finalized 2024), add as training pairs, compare top-1 against `in-pre` on the same frozen test.
2. Campaigns label themselves. Spread labels across copies of a form letter.	Thousands of letters are copies or light edits of one template, and they all argue about the same section. If one copy cites § 106.44, every copy does too. That's our core campaign-detection work turned into labels. Rank passages as a graph (near-duplicates linked), not one at a time.	Label propagation over the near-duplicate graph we already build. Measure top-1 on test passages whose campaign has a citing copy.
3. The cascade trains its replacement.	Every unsure passage we send to Sonnet comes back with an answer. Those are exactly the hard cases the model gets wrong. Feed them back as training data, and each retrain needs Sonnet less. This is also the automated retraining MS5 asks for.	Simulate: Sonnet answers for flagged train passages → retrain → re-run the cascade. Track the share sent and cost per 1,000 per round.
4. Write the comments a new rule will get.	On a brand-new rule we have no letters. Have an LLM write, once per rule, dozens of comments per section in different voices (a school district, an advocacy group, a parent). Paid once per rule, not per passage. Builds on the preamble result (50% → 67%).	Held-out ED: preamble only vs preamble + synthetic comments. Same for the 5 rules Livia is running.
5. Link the ask, not the paragraph.	A passage often mentions several sections, but its request ("we urge the Department to withdraw…") points at one. Find the request sentence first, then link that.	Score top-1 linking only the request sentence vs the whole passage.
6. Answer with a set, not a guess.	Two of us agree only 70% of the time, so sometimes there isn't one right section. Conformal prediction returns the smallest set of sections guaranteed to contain the answer, e.g. 90% of the time. Often 1 section, 2 or 3 when it's genuinely ambiguous. An analyst can trust that.	Calibrate on validation, report coverage and average set size on test. Our top-5 already covers 93%.

Simpler ones we'd try first: a reranker that reads passage and section together for the top 3; Sonnet choosing from our top 5 instead of all 11 to 39 sections (shorter prompt, easier choice).

7 Where it's still weak

- Two rules are bad: CPP 46.7% and LCRI 52.5%. The others are 60 to 76%.
- The labels limit us. Two of us labelled the same 30 passages and agreed on only 21 (70%), so the ceiling is around 83%.
- The live endpoint still uses only the base model. The two extra signals from item 1 aren't served yet.
- We've only held out one rule. The other five haven't been tested as unseen.

8 Questions for Alyssa

- Staff on Ed said GPUs need the billing account upgraded to paid. Is CPU training on Vertex enough for MS3, or should we upgrade?
- The Cloud Run service is private. Does it need to be public for grading, or is staff access enough?
- Is it fine to lead with the cascade as our main result, or should we lead with the plain model vs. Sonnet?
- Do you want unseen-rule tests on the other five rules too, or is one enough?